SUP'COM
المدرسة العليا للمواصلات بتونس
Ecole Supérieure des Communications de Tunis
Higher School of Communications of Tunis

CLOUD OF THINGS PROJECT

GreenPilot

Connected Greenhouse Monitoring, Irrigation
and Visual Disease Tracking

Cahier des charges

Functional and Technical Specifications

Prepared by

Mohamed Ali El Filali
Mahdi Fakhfakh

Faculty advisor

MOHAMED BECHA Kaaniche

Contents

1 Concept **2**
1.1 Project Context . . . 2
1.2 Problem Statement . . . 2
1.3 Objectives . . . 2

2 Client **2**

3 Functional need **2**
3.1 Sensors and IoT network . . . 2
3.2 PWA application . . . 3

4 Machine learning integration **3**
4.1 Soil-moisture forecasting . . . 3
4.2 Computer vision for continuous disease monitoring . . . 3
4.3 Vision dataset and evaluation . . . 4
4.4 MLOps implementation . . . 4

5 Equipment **5**

6 Technologies choice **5**
6.1 Back-end . . . 5
6.2 Middleware . . . 5
6.3 Front-End . . . 5
6.4 IoT integration . . . 5
6.5 MLOps integration . . . 6

7 Architecture **6**

8 Deployment diagram **7**

9 Timeline and tasks **8**

10 Methodology **8**
10.1 Principles . . . 8
10.2 Process . . . 8

11 Limitations **9**

12 Business study **10**
12.1 Business Model Canvas (BMC) . . . 10
12.2 Marketing policy . . . 10

1 Concept

1.1 Project Context

Small greenhouse operators require consolidated information on growing conditions, irrigation activity and visible plant-health symptoms. GreenPilot integrates physical measurements and leaf images with a cloud application to support remote supervision and traceable interventions.

The system combines an ESP32 acquisition and control node, a Raspberry Pi edge gateway, an ESP32-CAM camera node and cloud services. Two bell pepper plants represent two monitored zones. A shared irrigation circuit serves both zones. The gateway performs soil-moisture forecasting and automatic, repeated YOLO leaf detection with grower alerts; the cloud maintains histories, access control and location-based services.

1.2 Problem Statement

The project addresses four needs: heterogeneous data acquisition, bounded remote irrigation, early identification of conditions requiring inspection, and geographic access to relevant equipment. Measurements and visual observations must remain associated with their source, plant, zone and acquisition time.

1.3 Objectives

1. Monitor air temperature, relative air humidity, soil-moisture indicators and reservoir availability.
2. Control a shared irrigation circuit with local protection and command acknowledgement.
3. Forecast each zone's soil-moisture indicator 30 minutes ahead.
4. Continuously monitor identified bell pepper plants, detect and classify leaves in sampled images and alert the grower to suspected disease.
5. Provide grower and administrator PWAs backed by Jakarta EE 11 and MongoDB.
6. Describe connected equipment using W3C Web of Things interfaces.
7. Locate accessible greenhouses using equipment coordinates and the user's position.

System boundary. The delivered system covers one physical greenhouse demonstrator, two monitored zones and one shared pump. Visual detection supports inspection; it does not establish a definitive diagnosis or prescribe treatment. Irrigation is governed by sensor-based rules and explicit operating limits.

2 Client

The intended client is a greenhouse operator, nursery or agricultural laboratory. The **grower** monitors plants and equipment, requests irrigation and reviews disease alerts. The **administrator** manages accounts, sites, devices, control parameters and model deployments.

3 Functional need

3.1 Sensors and IoT network

- **F01 – Acquisition:** collect soil and ambient measurements every 60 seconds with device, site, zone, timestamp, unit and quality status.
- **F02 – Irrigation:** accept bounded manual requests and local threshold-based requests. Enforce reservoir availability, maximum duration, cooldown and a default-off state on restart. Reject expired or duplicate commands.
- **F03 – Continuity:** buffer records during cloud disconnection and synchronize without duplication after recovery. Local rules and inference shall remain available while the gateway operates.
- **F04 – Communication:** the ESP32-WROOM exchanges JSON measurements and commands with the Pi over a persistent local secure WebSocket (WSS) connection; HTTPS provides configuration/status endpoints. The Pi communicates with the cloud exclusively through the MQTT/TLS broker, including telemetry, commands, alerts and evidence images. Expose WoT properties, actions and events with documented bindings.
- **F05 – Continuous visual monitoring:** run an automatic camera acquisition and YOLO detection loop on the Raspberry Pi. Sample every 30 seconds initially, with a configurable interval. Detect leaves automatically; associate detections with fixed plant zones configured during installation; no grower

request is required for routine monitoring.

3.2 PWA application

F06 – Grower interface: provide current values, histories, forecasts, equipment state, bounded irrigation requests, alert acknowledgement and visual inspection timelines. Each inspection shows its image with detected boxes, capture time, predicted class, model score, model version and review status.

F07 – Administration: manage users and permissions, site/zone/plant identifiers, device descriptions, thresholds and versioned model releases. A review may confirm or reject a suspected condition without overwriting the original inference.

F08 – Location-based service: store greenhouse coordinates and equipment-to-zone associations. With permission, use phone geolocation to query nearby accessible sites. Display distance and active incidents.

F09 – Disease alerts and tracking: after repeated qualifying positive results for the same plant, open an incident and alert the grower in the PWA. Retain the evidence image, time, zone, model score and version. Deduplicate alerts while the incident remains open; support acknowledgement, review notes and closure. Scores do not measure disease severity.

Data sources. Soil and air sensors, reservoir status, camera images, phone position, operator commands and review annotations form distinct input streams. All displayed observations shall expose freshness; cached values shall be labelled when offline.

4 Machine learning integration

4.1 Soil-moisture forecasting

The forecasting model estimates the relative soil-moisture indicator at $t + 30$ minutes. Its output supports operator decisions and does not directly activate the pump.

Element	Specification
Inputs	Current and lagged moisture, recent trend, temperature, air humidity and time since watering.
Dataset	Timestamped measurements from repeated watering and drying cycles across both zones; acquisition begins in week 2. At least ten distinct cycles are targeted.
Reference	Persistence baseline: $\hat{m}_{t+30} = m_t$. Compare linear regression and a compact tree-based regressor.
Evaluation	MAE and RMSE on later held-out cycles. Keep overlapping windows from the same cycle together; fit preprocessing only on training data. Report results by zone.
Release gate	Select a predictive model only if held-out MAE improves on persistence. Otherwise retain the baseline and document the evaluation.

Each soil probe shall be normalized using documented reference conditions. Readings are relative indicators, not volumetric water content. Stale or invalid inputs shall suppress inference. Watering after prediction shall be recorded because it changes the subsequent observed moisture.

4.2 Computer vision for continuous disease monitoring

Scope. A YOLO object detector locates bell pepper leaves and assigns two classes: healthy leaf or leaf with suspected bacterial spot. Each result contains a bounding box, class and confidence score. Failed quality checks, no valid leaf detection or insufficient confidence yield "inconclusive". Other diseases and crops are outside the trained scope; scores cannot establish that an input is in scope.

Automatic monitoring loop. The ESP32-CAM provides local HTTPS JPEG snapshots; a USB webcam plugged into the Pi is an alternative. Every 30 seconds, the Pi checks image quality and runs YOLO on the frame. Configure two non-overlapping plant zones at installation; associate detected leaves by box centre, rejecting ambiguous zone assignments. YOLO finds the leaf boxes automatically, without manually cropping individual leaves. The service starts with the gateway and exposes observation time and status. Process one frame at a time; continuous monitoring does not require every video frame.

Detection and alert rule. A qualifying bacterial-spot box in a plant zone marks that observation as *disease suspected*. If no disease box qualifies but a healthy-leaf box does, report *healthy observation*;

otherwise report *inconclusive*. A healthy observation concerns visible detected leaves only. Three consecutive qualifying disease observations for the same plant open an incident; healthy or inconclusive observations reset an unconfirmed counter. Select confidence thresholds on validation data. Absence of a valid leaf detection shall be reported as inconclusive.

Grower notification. The Pi publishes the confirmed event and its JPEG evidence through the cloud MQTT/TLS broker. A backend subscriber correlates the messages, stores the image and incident in MongoDB, then sends an application acknowledgement to the Pi. The grower PWA retrieves its alert from the web API. While the PWA is open, refresh the alert list at most every 10 seconds. Browser push notifications may be enabled with permission; background delivery is an extension. Repeat detections update the open incident rather than sending a new alert every cycle. The grower acknowledges and reviews the incident before closing it.

History and storage. Record plant, zone, time, detected boxes, classes, confidence scores, model version and quality status. Retain incident evidence with box overlays, follow-up snapshots and routine healthy snapshots initially every 10 minutes. Preserve original box coordinates with image dimensions. Raw continuous video is not uploaded or archived.

Model. Fine-tune a pretrained YOLO nano model to detect healthy bell pepper leaves and leaves with suspected bacterial spot.

4.3 Vision dataset and evaluation

Training data. Build a reviewed detection dataset with a bounding box around each visible target leaf and a healthy/bacterial-spot label. Record provenance, dataset version and usage terms. PlantVillage bell pepper images may supplement training after manual box annotation; image-level class labels alone are insufficient for detector training. Add project-camera images with realistic backgrounds, multiple leaves and negative frames without target leaves. Review field disease labels; unverified images serve workflow demonstrations only. Store YOLO-format box labels and a dataset configuration file.

Split protocol. Separate training, validation and test data before augmentation. Group images of the same leaf or plant where identifiers are available, remove exact duplicates and inspect near-duplicates. Keep local capture sessions separate from training to assess transfer to the demonstrator. Preserve the test set for final evaluation.

Metrics. Report detection mAP@0.5 and mAP@0.5:0.95, per-class precision/recall, annotated class counts, missed leaves and false positives. Measure plant-observation macro-F1, inconclusive rate, Pi latency, memory and capture-to-display delay. Report local-camera results separately from public images. Test the temporal alert rule on labelled sequences, measuring false alerts, missed incidents and confirmation delay.

Release criteria. Report held-out detection metrics and compare plant-level decisions with a majority-class baseline. Release only after repeatable exported-model inference and an end-to-end alert test on the Pi. The configured sampling interval must accommodate measured processing time without an unbounded queue. Preserve model version and inconclusive outcomes; report observed accuracy and coverage without assuming generalization.

4.4 MLOps implementation

1. Version the sensor and image datasets, split manifests, labels and preprocessing configurations.
2. Train both models on the project PC in dedicated training sessions. The training environment is separate from the live inference services; training is not required during operation.
3. Package model files, preprocessing, class/feature schemas, metrics, dataset references and checksums in versioned releases.
4. Deploy validated releases to the gateway, run a smoke test and record the active version. Maintain the previous release for rollback.
5. Log inference outputs and reviewer feedback. Review forecast errors and classification performance before initiating retraining; incoming observations shall not automatically alter deployed models.

5 Equipment

The table identifies the essential electronic and electrical equipment. The selected models shall be compatible with 3.3 V control signals and the pump's rated supply.

Equipment	Qty.	Function
Raspberry Pi 4 Model B, 4 GB	1	Dedicated edge gateway: Node-RED, MQTT client, local buffering, forecasting and visual inference.
ESP32-WROOM development board	1	Sensor acquisition, local HTTPS/WebSocket communication and protected pump control.
Capacitive soil-moisture probe	2	One relative soil-moisture measurement per zone.
Air temperature/humidity sensor	1	Ambient measurements for the greenhouse.
Float-type reservoir level switch	1	Detect insufficient water and inhibit pump activation.
Low-voltage DC water pump	1	Small electrical pump serving the shared irrigation circuit; 5 V or 12 V according to the selected model.
3.3 V-compatible MOSFET driver	1	Electronic power switch controlled by the ESP32; include inductive-load protection.
ESP32-CAM camera board	1	Primary camera: local HTTPS JPEG snapshots for continuous monitoring on the Pi. Include a compatible programming interface.
Electrical wiring and power cables	1 set	Sensor/driver connections and power; runtime node communication is wireless.

6 Technologies choice

6.1 Back-end

MongoDB stores sites, GeoJSON coordinates, devices, plants, measurements, irrigation history, incidents, images, predictions and model versions. Inspection records reference image identifiers. A `2dsphere` index supports proximity queries; access is mediated by the API.

MQTT / Eclipse Mosquitto carries all gateway–cloud messages over TLS: measurements, commands, predictions, alerts and JPEG evidence. The Pi and backend are broker clients. Commands have identifiers and expiry times; commands and image messages are not retained.

6.2 Middleware

Jakarta EE 11 / Jakarta REST implements authentication, roles, queries, image retrieval, location search, control auditing and inspection workflows. Deploy its WAR on **WildFly**. **Nginx** serves the PWAs over HTTPS and proxies `/api` to WildFly privately, providing one public web address. PWA HTTPS access is separate from the broker-only gateway connection.

6.3 Front-End

Grower and administrator **PWAs** share HTML/CSS/JavaScript components. A manifest and service worker support installation and cached assets. Live controls require connectivity; geolocation requires permission. Plant images come from the gateway camera.

6.4 IoT integration

Node-RED receives timestamped ESP32 JSON through local secure WebSocket (WSS), applies rules, buffers data and exchanges cloud MQTT messages. Commands return over WSS; HTTPS serves local status/configuration. The ESP32-CAM provides HTTPS JPEG snapshots; a USB webcam plugged directly into the Pi is an alternative. Use authenticated TLS endpoints with provisioned certificates. Local services continue during cloud outages while the LAN remains available; firmware enforces pump protections.

W3C WoT Thing Descriptions expose measurements and pump state as properties, bounded watering and monitoring start/stop as actions, and low-water or confirmed-disease alerts as events. Declare actual protocol bindings, schemas and security, and validate descriptions against implemented endpoints.

6.5 MLOps integration

Python/pandas/scikit-learn support forecasting; **Ultralytics YOLO/PyTorch** trains a nano leaf detector; **OpenCV** handles images. Versioned forecasting and NCNN-exported YOLO artifacts run on the Pi. Git and dataset/model manifests track reproducible releases. CI checks builds, API contracts and representative inference cases.

7 Architecture

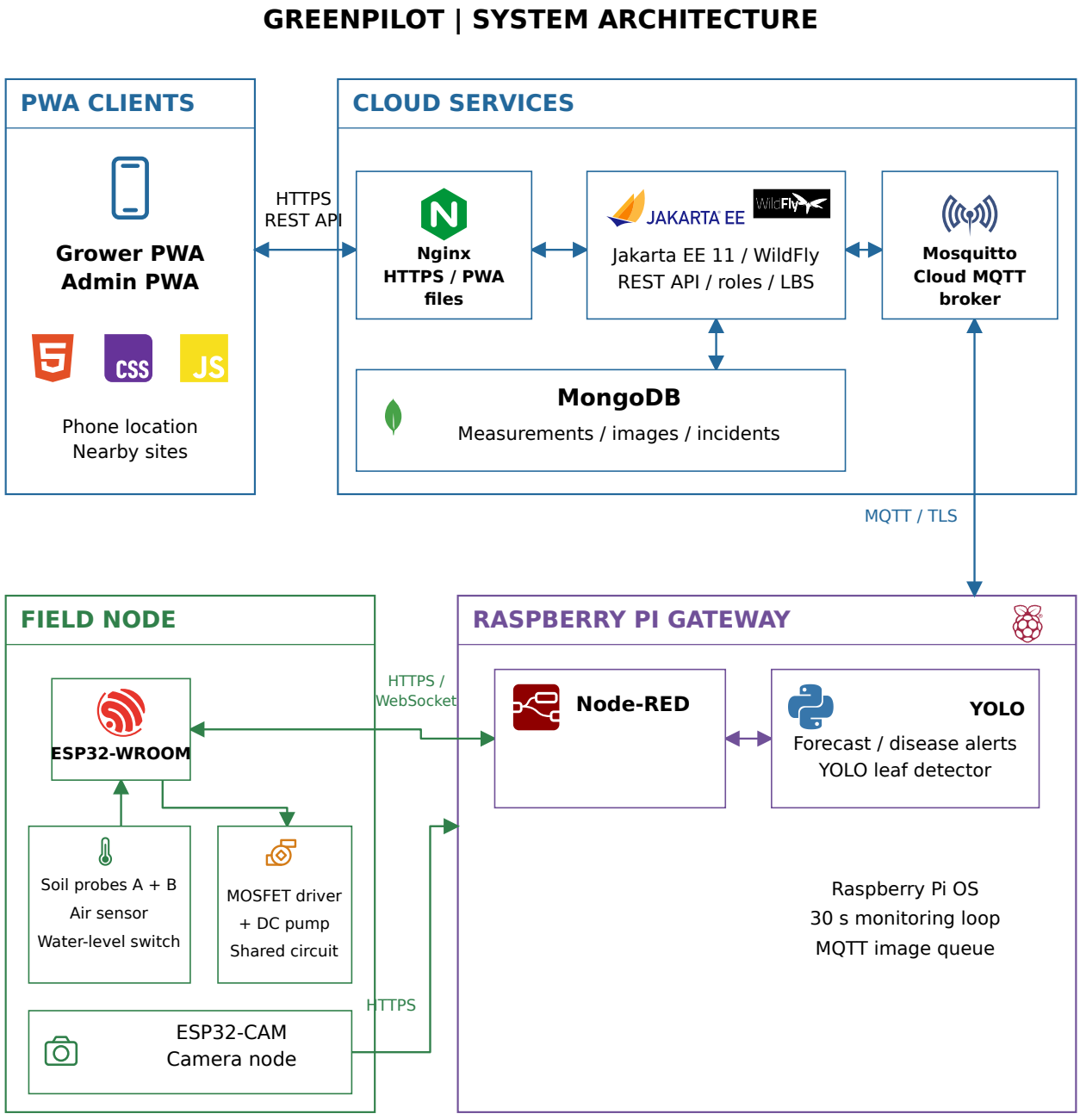


Figure 1: GreenPilot architecture

8 Deployment diagram

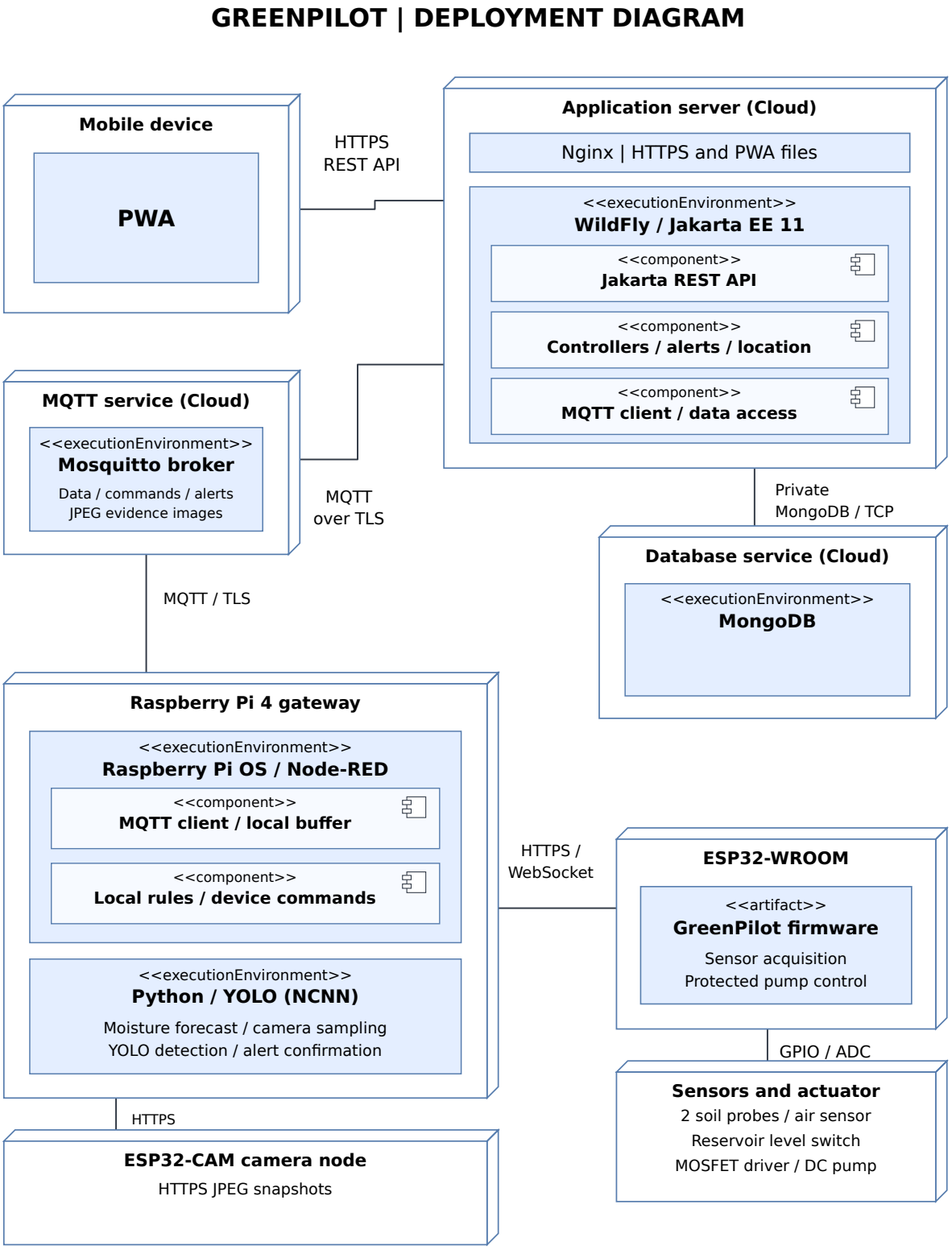


Figure 2: GreenPilot deployment. Application, MQTT and database services are hosted in the Cloud on one VM.

9 Timeline and tasks

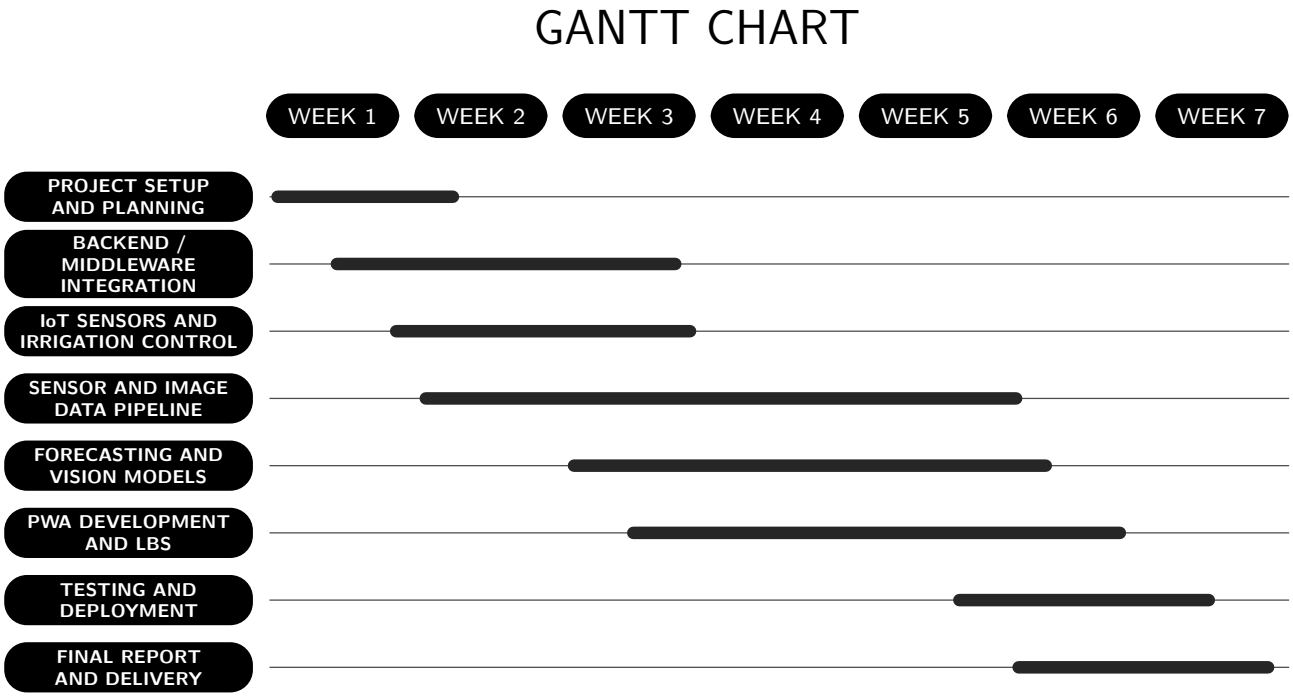


Figure 3: Seven-week schedule: overlapping implementation, data collection, model development and integration.

Milestones. W1: approved requirements and dataset scope. W2: measurements and camera capture available. W4: wireless local control and MQTT evidence delivery. W5: evaluated forecasting and vision releases. W6: integrated WoT, location and outage tests. W7: acceptance report and complete delivery.

10 Methodology

The project follows **Extreme Programming (XP)** with weekly iterations, shared ownership and demonstrable increments.

10.1 Principles

- **Simplicity:** one irrigation circuit, one forecasting horizon and a two-class vision task.
- **Communication:** maintain a shared backlog, interface contracts and weekly supervisor reviews.
- **Pair programming:** jointly review actuator logic, dataset splits, image processing and integration interfaces.
- **Feedback:** demonstrate each increment and record measured outcomes.
- **Quality:** maintain reproducible builds and check failure paths as well as nominal behaviour.

10.2 Process

1. **Planning:** select user stories, dependencies and acceptance criteria.
2. **Design:** update UML deployment, data schemas, WoT descriptions and service contracts.
3. **Implementation:** develop firmware, gateway services, models, APIs and PWA features in reviewed increments.
4. **Verification:** execute integration tests, model evaluations and hardware scenarios against the specified criteria.
5. **Review:** present the increment to the supervisor, record findings and prioritize the next iteration.

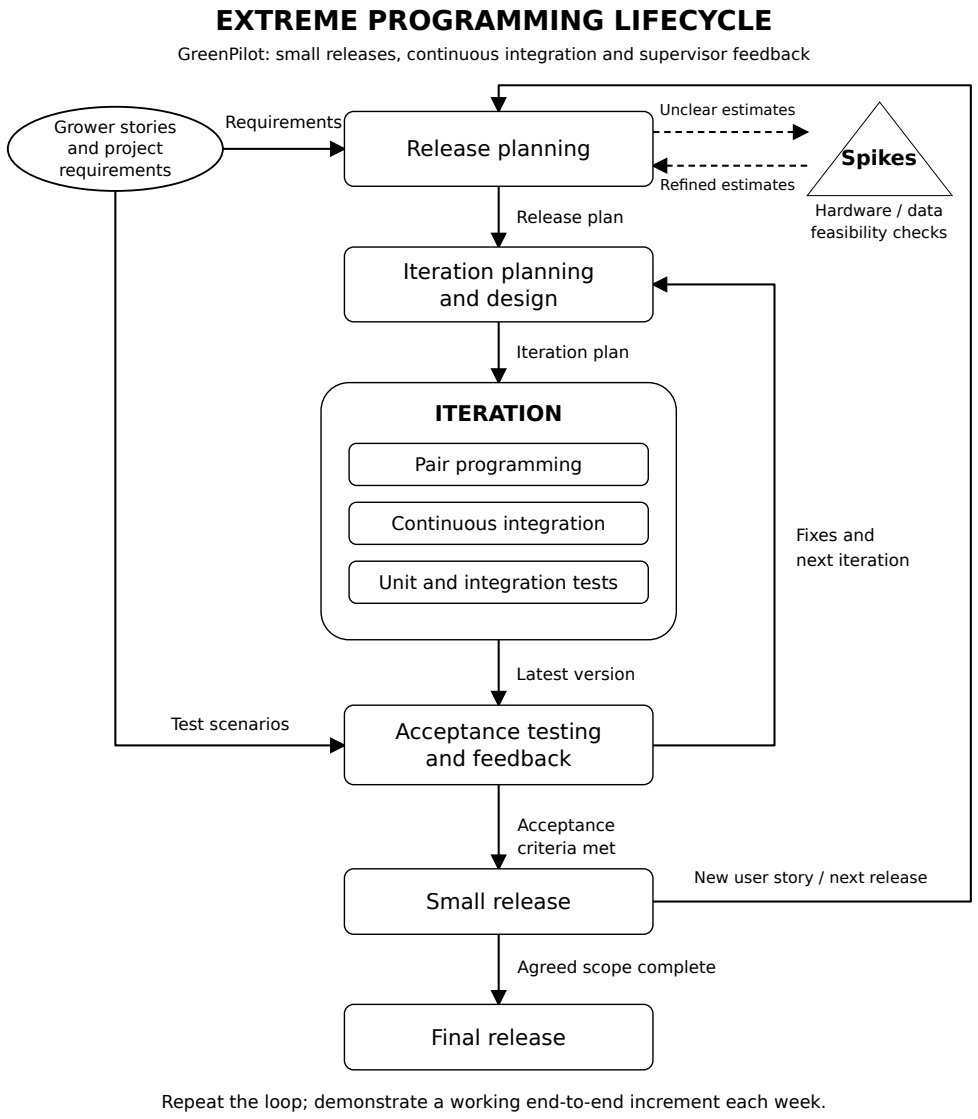


Figure 4: Extreme Programming lifecycle: release planning, short development iterations, acceptance testing and feedback loops.

11 Limitations

- **Prototype scale:** one greenhouse, two zones and one shared pump; independent zone irrigation is not supported.
- **Soil measurements:** probes provide calibrated relative indicators, affected by substrate, placement and sensor variability.
- **Forecasting:** limited watering/drying cycles restrict generalization; evaluate against persistence on separate cycles.
- **Disease coverage:** only healthy bell pepper leaves and suspected bacterial spot are supported; other crops and diseases are outside the trained scope.
- **Local communication:** network loss interrupts node exchanges. MQTT image delivery requires bounded queues and compatible packet limits.
- **Camera coverage:** leaf detection is affected by occlusion, small leaves, movement, lighting and blur. Plant-zone assignment requires stable framing; absent detections remain inconclusive.
- **Dataset transfer:** public-image performance may not transfer to greenhouse images. Reviewed box annotations and local disease examples may be limited.
- **Alerts:** false or missed detections remain possible. Consecutive-positive confirmation delays alerts; grower review is required.

- **Availability:** cloud loss delays remote controls, images and alerts. Local services require the LAN, gateway and power; gateway failure interrupts inference and gateway automation.
- **Location:** GPS locates greenhouses, not adjacent pots; plant/zone identifiers are required indoors.
- **Operating scope:** definitive diagnosis, autonomous treatment, lesion severity, frame-by-frame tracking and production-scale multi-farm operation are excluded.

12 Business study

12.1 Business Model Canvas (BMC)

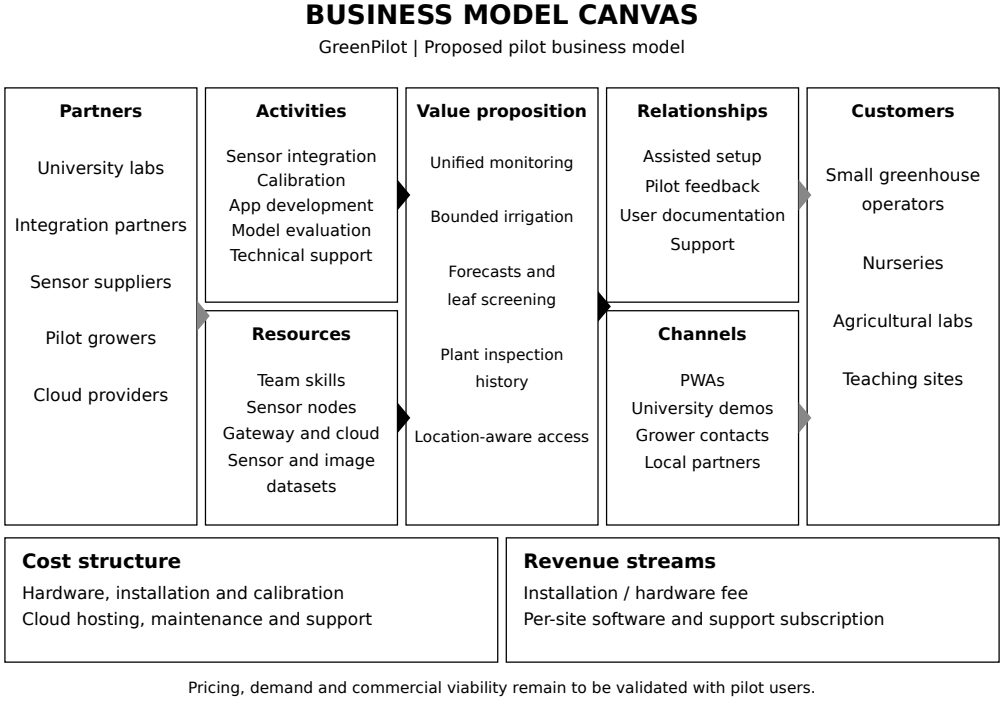


Figure 5: Business Model Canvas for the proposed GreenPilot pilot. Pricing and commercial assumptions require validation.

12.2 Marketing policy

Product. GreenPilot combines an acquisition node, a shared irrigation circuit, a camera, an edge gateway and cloud supervision. The grower PWA provides environmental histories, forecasts, continuous visual disease screening, automatic grower alerts and plant follow-up; the administrator PWA manages access and configuration.

Price. The proposed B2B model comprises a hardware/installation fee and a per-site software and support subscription. Pricing shall be established from the final bill of materials, installation effort, hosting requirements and maintenance obligations.

Promotion. Product demonstrations shall show the complete monitoring, irrigation and visual-inspection workflow. Communication shall use measured project results and clearly state the supported crop, classification scope and operating conditions.

Distribution. Software is distributed through HTTPS-hosted PWAs. Site deployment includes installation, sensor calibration, camera setup and user training. Agricultural-equipment partners and laboratories are potential distribution and support channels.